

Лекция по JavaScript (Часть 4)

Продолжительность: 3 часа

Предыдущие темы:

1. Введение в JavaScript: синтаксис, переменные, типы данных, операторы.
 2. Управляющие конструкции: условия, циклы, функции.
 3. Массивы, объекты, работа с DOM, асинхронность.
-

План лекции:

- 1. Модули в JavaScript (45 минут)**
 - Что такое модули?
 - Экспорт и импорт: `export`, `import`.
 - Практические примеры.
 - 2. Замыкания и области видимости (45 минут)**
 - Лексическое окружение.
 - Замыкания: теория и примеры.
 - Практическое применение замыканий.
 - 3. Классы и ООП в JavaScript (1 час)**
 - Введение в ООП.
 - Классы: синтаксис, конструктор, методы.
 - Наследование и статические методы.
 - Практические примеры.
 - 4. Перерыв (10 минут)**
 - 5. Практика и закрепление материала (1 час)**
 - Решение задач на модули, замыкания и классы.
 - Ответы на вопросы.
-

Подробное содержание лекции с комментариями:

1. Модули в JavaScript (45 минут)

Теория:

Модули позволяют разбивать код на отдельные файлы, что упрощает поддержку и организацию кода. В JavaScript модули появились с введением стандарта ES6.

Основные концепции:

- **Экспорт (`export`)**: Позволяет сделать переменные, функции или классы доступными в других модулях.
- **Импорт (`import`)**: Позволяет использовать экспортированные элементы из других модулей.

Пример 1: Экспорт и импорт по умолчанию

```
// math.js
// Экспортируем функцию add по умолчанию
export default function add(a, b) {
  return a + b;
}

// main.js
// Импортируем функцию add из модуля math.js
import add from './math.js';

// Используем импортированную функцию
console.log(add(2, 3)); // 5
```

Комментарии:

- В `math.js` мы экспортируем функцию `add` по умолчанию с помощью `export default`.
- В `main.js` мы импортируем эту функцию с помощью `import add from './math.js'`.
- После импорта функция `add` доступна для использования.

Пример 2: Именованный экспорт и импорт

```
// math.js
// Экспортируем несколько функций с именованным экспортом
export function add(a, b) {
  return a + b;
}
```

```
export function subtract(a, b) {
    return a - b;
}

// main.js
// Импортируем функции add и subtract из модуля math.js
import { add, subtract } from './math.js';

// Используем импортированные функции
console.log(add(5, 3)); // 8
console.log(subtract(5, 3)); // 2
```

Комментарии:

- В `math.js` мы экспортируем функции `add` и `subtract` с помощью именованного экспорта (`export function ...`).
 - В `main.js` мы импортируем эти функции с помощью `import { add, subtract } from './math.js'`.
 - Именованный экспорт позволяет импортировать только нужные функции.
-

2. Замыкания и области видимости (45 минут)

Теория:

Лексическое окружение:

- Это внутренняя структура JavaScript, которая хранит информацию о переменных и функциях в текущей области видимости.

Замыкания:

- Это функция, которая запоминает своё лексическое окружение даже после того, как выполнение кода вышло за пределы этой области видимости.

Пример 1: Простое замыкание

```
function createCounter() {
    let count = 0; // Локальная переменная count

    // Возвращаем функцию, которая увеличивает count на 1
    return function() {
```

```

        count++;
        return count;
    };
}

// Создаём счётчик
const counter = createCounter();

// Используем счётчик
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3

```

Комментарии:

- Функция `createCounter` возвращает другую функцию, которая имеет доступ к переменной `count`.
- Переменная `count` сохраняется в лексическом окружении, даже после завершения выполнения `createCounter`.
- Каждый вызов `counter()` увеличивает значение `count` на 1.

Пример 2: Практическое применение замыканий (приватные переменные)

```

function createPerson(name) {
    let age = 0; // Приватная переменная age

    // Возвращаем объект с методами для работы с age
    return {
        getName: () => name, // Метод для получения имени
        getAge: () => age,    // Метод для получения возраста
        setAge: (newAge) => age = newAge, // Метод для изменения возраста
    };
}

// Создаём объект person
const person = createPerson("John");

// Используем методы объекта
console.log(person.getName()); // John
person.setAge(30);
console.log(person.getAge()); // 30

```

Комментарии:

- Переменная `age` является приватной, так как она недоступна напрямую извне.
 - Мы можем изменять и получать значение `age` только через методы `getAge` и `setAge`.
-

3. Классы и ООП в JavaScript (1 час)

Теория:

ООП (Объектно-ориентированное программирование):

- Это парадигма программирования, основанная на объектах, которые содержат данные и методы для работы с этими данными.

Классы:

- Это шаблон для создания объектов. Классы в JavaScript появились в ES6.

Пример 1: Создание класса

```
// Создаём класс Person
class Person {
  // Конструктор вызывается при создании нового объекта
  constructor(name, age) {
    this.name = name; // Свойство name
    this.age = age;   // Свойство age
  }

  // Метод greet
  greet() {
    console.log(`Hello, my name is ${this.name}`);
  }
}

// Создаём объект person
const person = new Person("John", 30);

// Используем метод greet
person.greet(); // Hello, my name is John
```

Комментарии:

- Класс `Person` имеет конструктор, который инициализирует свойства `name` и `age`.

- Метод `greet` выводит приветствие с именем.

Пример 2: Наследование

```
// Создаём класс Student, который наследует Person
class Student extends Person {
  // Конструктор Student
  constructor(name, age, grade) {
    super(name, age); // Вызов конструктора родительского класса
    this.grade = grade; // Новое свойство grade
  }

  // Метод study
  study() {
    console.log(`${this.name} is studying`);
  }
}

// Создаём объект student
const student = new Student("Alice", 20, "A");

// Используем методы
student.greet(); // Hello, my name is Alice
student.study(); // Alice is studying
```

Комментарии:

- Класс `Student` наследует свойства и методы класса `Person`.
- Метод `super` вызывает конструктор родительского класса.
- Класс `Student` добавляет новое свойство `grade` и метод `study`.

Пример 3: Статические методы

```
// Создаём класс MathUtils
class MathUtils {
  // Статический метод add
  static add(a, b) {
    return a + b;
  }
}

// Используем статический метод
console.log(MathUtils.add(2, 3)); // 5
```

Комментарии:

- Статические методы принадлежат классу, а не его экземплярам.
 - Они вызываются через имя класса, а не через объект.
-

4. Практика и закрепление материала (1 час)

Задачи:

1. Создайте модуль `geometry.js`, который экспортирует функции для вычисления площади и периметра фигур.
2. Используйте замыкания для создания кэширующей функции, которая запоминает результаты вычислений.
3. Создайте класс `BankAccount` с методами для внесения и снятия денег, а затем класс `SavingsAccount`, который добавляет процентный доход.

Пример решения:

```
// geometry.js
// Экспортируем функции для работы с геометрией
export function calculateRectangleArea(width, height) {
    return width * height;
}

export function calculateRectanglePerimeter(width, height) {
    return 2 * (width + height);
}

// main.js
// Импортируем функции из geometry.js
import { calculateRectangleArea, calculateRectanglePerimeter } from
    './geometry.js';

// Используем функции
console.log(calculateRectangleArea(5, 10)); // 50
console.log(calculateRectanglePerimeter(5, 10)); // 30

// Замыкания: кэширующая функция
function createCachedFunction(fn) {
    const cache = new Map(); // Хранилище для кэша

    return function( ...args ) {
```

```

    const key = JSON.stringify(args); // Ключ для кэша

    if (cache.has(key)) {
        console.log("Returning cached result");
        return cache.get(key);
    }

    const result = fn( ... args); // Вычисляем результат
    cache.set(key, result); // Сохраняем результат в кэше
    return result;
};
}

// Пример использования
const cachedAdd = createCachedFunction((a, b) => a + b);
console.log(cachedAdd(2, 3)); // 5 (вычислено)
console.log(cachedAdd(2, 3)); // 5 (возвращено из кэша)

// Классы: BankAccount и SavingsAccount
class BankAccount {
    constructor(balance = 0) {
        this.balance = balance;
    }

    deposit(amount) {
        this.balance += amount;
        console.log(`Deposited ${amount}. New balance: ${this.balance}`);
    }

    withdraw(amount) {
        if (amount > this.balance) {
            console.log("Insufficient funds");
            return;
        }
        this.balance -= amount;
        console.log(`Withdrawn ${amount}. New balance: ${this.balance}`);
    }
}

class SavingsAccount extends BankAccount {
    constructor(balance = 0, interestRate = 0.01) {
        super(balance);
        this.interestRate = interestRate;
    }

    addInterest() {

```



```
        const interest = this.balance * this.interestRate;
        this.deposit(interest);
        console.log(`Interest added: ${interest}`);
    }
}
```

// Использование

```
const savings = new SavingsAccount(1000, 0.02);
savings.addInterest(); // Interest added: 20
savings.withdraw(500); // Withdrawn 500. New balance: 520
```

Итог:

- **Модули** помогают организовать код.
- **Замыкания** позволяют сохранять состояние между вызовами функций.
- **Классы** и **ООП** упрощают создание сложных структур данных.

Домашнее задание:

1. Модули в JavaScript (не выполнять)

Задание:

Создайте модуль `geometry.js`, который экспортирует функции для вычисления площади и периметра **разных фигур** (например, прямоугольника, круга, треугольника).

- Добавьте функцию для вычисления площади круга.
- Добавьте функцию для вычисления периметра треугольника.

2. Замыкания

Задание:

Используйте замыкания для создания функции-генератора уникальных идентификаторов (ID).

- Каждый вызов функции должен возвращать новый уникальный ID (начиная с 1).
- Добавьте возможность сброса счётчика ID.

3. Классы и ООП

Задание:

Создайте класс `Animal` с методом `sound`, который выводит звук животного.

- Создайте классы `Dog`, `Cat` и `Bird`, которые наследуют `Animal` и переопределяют метод `sound`.
- Добавьте в класс `Animal` свойство `name` и метод `introduce`, который выводит: "My name is [name], and I say [sound]".

Дополнительные задания (по желанию):

4. Модули и замыкания

Задание:

Создайте модуль `cache.js`, который экспортирует функцию для кэширования результатов любой другой функции.

- Функция должна принимать другую функцию в качестве аргумента и возвращать новую функцию, которая кэширует результаты.
- Используйте замыкания для хранения кэша.

5. Классы и ООП

Задание:

Создайте класс `BankAccount` с методами для внесения и снятия денег.

- Добавьте свойство `transactionHistory`, которое хранит историю всех операций (внесение, снятие).
- Создайте класс `SavingsAccount`, который наследует `BankAccount` и добавляет метод для начисления процентов.

Итог:

- **Модули:** Студенты научатся разбивать код на модули и экспортировать/импортировать функции.
 - **Замыкания:** Поймут, как использовать замыкания для создания функций с сохранением состояния.
 - **Классы и ООП:** Освоят основы ООП, включая наследование и переопределение методов.
-

Следующая лекция:

- Работа с API: `fetch`, `axios`.
- Обработка ошибок в асинхронном коде.
- Продвинутые темы: итераторы, генераторы, прокси.